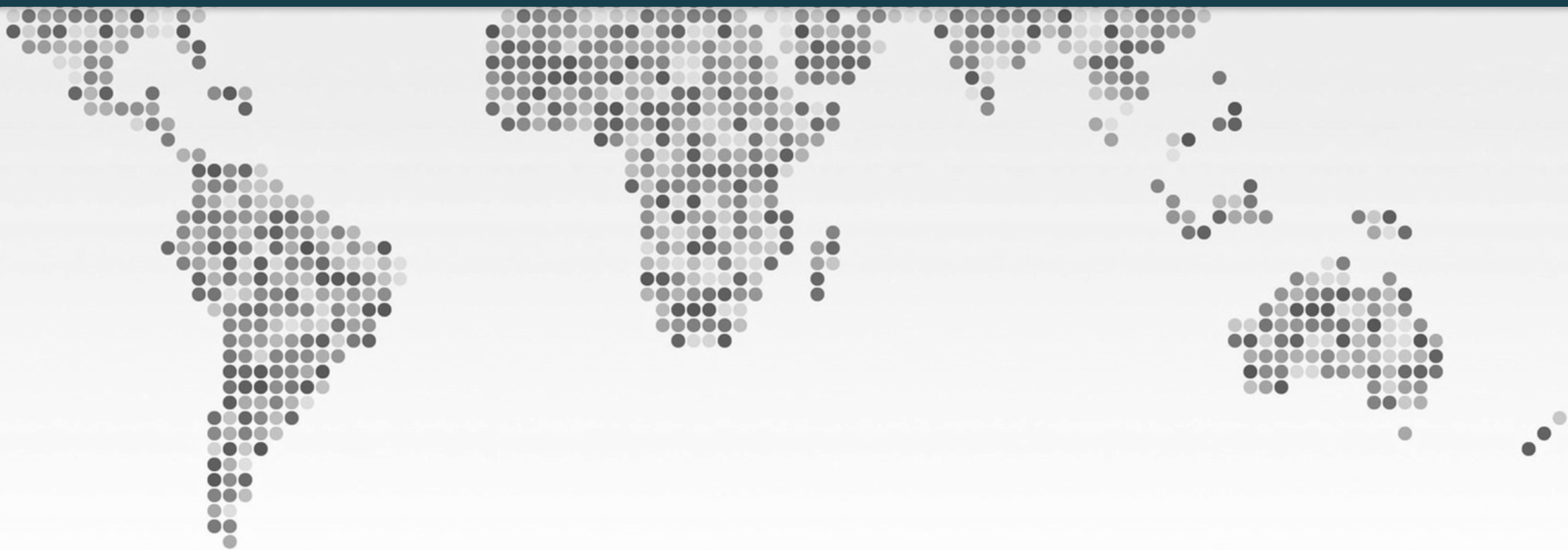


Software Engineering for Artificial Intelligence

Testing and Experimenting in Production



Software Engineering for Artificial Intelligence: Testing in Production

Testing and Experimenting in Production

With model validation and testing, we still refer to **offline** validation and verification—that is, the evaluation of models using held-out datasets and input testing strategies *prior* to deployment.

This process allows us to assess correctness in a *controlled/artificial setting*. However, such testing is inherently limited: it assumes that future inputs will resemble those seen during offline settings. To address this gap, we now move toward **testing in production**, which aims to *monitor and assess model behavior under real-world conditions*, where unexpected inputs, data drift, or unseen failure modes may emerge.

Testing in production. The process of evaluating a model's behavior after deployment, using real-world inputs and conditions. It complements offline validation by capturing issues like data drift, unexpected inputs, or performance degradation that only emerge in live environments.

The overarching idea is to “try” ideas directly in production multiple times a day to see what works, receiving real, representative data from the real environment.

Testing in production can avoid the common pitfalls of offline model evaluations, such as test data not being representative of production data. However, testing in production also raises new challenges, since **quality problems may be directly exposed to users**, potentially causing real harm.

Software Engineering for Artificial Intelligence: Testing in Production

Testing and Experimenting in Production

Testing in production is not a new idea and has been used in systems without AI components for a long time; yet, it seems to be particularly relevant for AI systems, as their behavior depends on data, which can change over time in unpredictable ways.

Some of you might have heard already of *alpha tests* and *beta tests*, which were traditional ways of getting users to test a product before its release—either at very early stages (alpha testing) and at a near-release stage (beta testing).

Users participating in alpha and beta testing are *usually aware* that they are using a pre-release version of the software and should expect some problems.

In pre-internet days, companies would recruit hundreds or thousands of volunteers who would use early versions of the product and hopefully report bugs and answer surveys. For instance, fifty thousand beta testers helped with *Windows 95*, many of whom paid \$20 for the privilege. Today, feedback is often collected through **telemetry** and **online forms**. Many start-ups provide only limited access to products still in development; for example, OpenAI initially had waitlists for DALL·E and ChatGPT.

The first big telemetry step was sending **crash reports** whenever the software crashed. A management component would detect the crash and collect additional information, such as stack traces and configuration options, and upload them to a server of the developing company. Soon, many software systems started collecting and uploading telemetry not only about crashes but about all kinds of **usage behavior**.

Software Engineering for Artificial Intelligence: Testing in Production

Testing and Experimenting in Production

Firefox Data Collection and Use

We strive to provide you with choices and collect only what we need to provide and improve Firefox for everyone. We always ask permission before receiving personal information.

[Privacy Notice](#)

- ☒ Allow Firefox to send technical and interaction data to Mozilla [Learn more](#)
- ☒ Allow Firefox to make personalized extension recommendations [Learn more](#)
- ☒ Allow Firefox to install and run studies [View Firefox studies](#)
- ☐ Allow Firefox to send backlogged crash reports on your behalf [Learn more](#)

For instance, many desktop applications provide consent or opt-out mechanisms for sharing data about user behavior with developers.

Software Engineering for Artificial Intelligence: Testing in Production

Testing and Experimenting in Production: Telemetry, Again

As already discussed when talking about observability, telemetry data is inherently continuous, and analyses will usually be time-series analyses that monitor trends in quality measures over time.

A solid telemetry system for monitoring system and model quality in real time provides also the foundation for experimenting in production. Rather than “just” observing the system over time, we can **intentionally introduce changes in the system to observe how users react to those changes**.

Experimentation in production can be very powerful and identify surprising and counterintuitive designs that developers may not have anticipated.

For example, Microsoft described how presenting ads on Bing differently—by including more text in the link—increased ad revenue by 12 percent or 100 million US dollars per year, without affecting other measures of user engagement.

Developers did not anticipate the strong effect and, in fact, did not implement the change for several months after it was initially suggested. This was one of thousands of experiments done that year, of which only a few showed strong results.

Hence, rather than relying on developers’ intuition to accurately predict what influences user behavior, experimenting in production can explore choices at scale.

Testing and experimenting in production is pretty popular, and most systems with AI components are tested in production at some point.

Software Engineering for Artificial Intelligence: Testing in Production

Testing and Experimenting in Production: A/B Experiments

Classic human-subject experiments in psychology or medicine divide participants into two groups, giving one group the treatment and using the other group as a control group, to then compare whether there are differences in outcomes between the groups.

Running traditional experiments in psychology or medicine is challenging, because it is usually difficult and expensive to recruit enough participants and it is important but non-trivial to ensure that the two groups are actually comparable—so that we do not have more healthy participants in one group of a medical trial.

Cloud-based and internet-connected services with many users provide **perfect conditions to run controlled experiments**. We can divide users already using our product into two groups and show them *different versions* of the product.

Since we are experimenting on existing users, we do not have *additional recruitment costs* and we may have *thousands or millions of participants*. The ethics of experimenting on users is debated, because companies usually do not seek consent, at most inform users with vague privacy policies, and rarely provide opt-out options. There is no government oversight comparable to research ethics in academia. Regardless, **such experimentation is a very common practice**.

With enough users, it becomes feasible to run hundreds of experiments at the same time, as many big-tech companies do these days. We can constantly experiment, validate new features rapidly in production, and make many design decisions with insights from experiments

Software Engineering for Artificial Intelligence: Testing in Production

Testing and Experimenting in Production: A/B Experiments

This is another example of the *Flywheel effect*: The more users, the easier it is to experiment in production, and more experiments help improve the product, which attracts more users.

Controlled experiments in production are called **A/B experiments**.

A/B Experiment (or A/B Testing). A method used to compare two versions of a system, called *Version A* (the control) and *Version B* (the variation), by exposing different user groups to each version and measuring which performs better based on predefined metrics (e.g., accuracy, user engagement, error rate).

In other terms, we simply assign some percentage of the users to a **treatment** group and serve them a different variant of the product, typically differing only in a small change, e.g., trying a different prompt for generating meeting minutes or presenting the meeting minutes with a different layout in a automated minute transcription system.

For example, not all Whatsapp users received a feature allowing them to chat with Meta AI (Llama 3.2): this might be an example of an A/B experiment meant to analyze the user engagement with the feature before distributing it on a large scale.

To implement A/B experiments, we need three ingredients: (1) two or more alternative implementation variants to be tested, (2) a mechanism to map users to those variants, and (3) a mechanism to collect telemetry separately for each variant.

Software Engineering for Artificial Intelligence: Testing in Production

Testing and Experimenting in Production: A/B Experiments

- 1 *Implementing variants.* To serve two alternative variants, we can either deploy two separate variants and decide which users to route to which variant at the network level with a load balancer, or we can encode the differences as a simple control-flow decision within a single application—a practice called **feature flags**.

Feature flags are simply boolean options that are used to decide between two control flow paths in the implementation. Multiple feature flags can control differences in the same system. Ideally, feature flags are tracked explicitly, documented, and removed once the experiment is done.

```
if (features.enabled(userId, "new_model_experiment5")) {  
    // new feature extraction  
    // predict with new model and new features  
} else {  
    // old feature extraction  
    // predict with old model  
}
```

An example of a feature flag used to decide between two models. Feature flags are usually enabled for specific users.

Testing and Experimenting in Production: A/B Experiments

- 2 *Mapping users to variants.* Next, we need to decide which users will see which variant. Random assignment of users to variants is usually a good choice to avoid bias in how groups are formed, for example, where one group contains all the most active users.

It is also usually a good idea to assign users to groups in a stable fashion, so that they do not see different variants whenever they reload the web page.

Simple functions are usually sufficient, like `function isEnabled(userId) {return (hash(userId)%100)<10}`, which selects 10 percent of all users in a stable fashion.

It is also possible to stratify the sample by randomly selecting from different subgroups, such as selecting fifty percent of beta-users, ninety percent of developers (dogfooding), and 0.1 percent of all normal users.

Please note that even when users cannot easily be tracked with accounts, it is worth trying to identify repeat users to keep them in the same group for repeated interactions, for example, based on their IP address.

- 3 *Telemetry.* Finally, it is important that telemetry and derived measures for system or model quality can be mapped to experimental conditions. That is, we need to know which actions were associated with users in each experimental condition.

Software Engineering for Artificial Intelligence: Testing in Production

Testing and Experimenting in Production: A/B Experiments - The Role of Statistics

Once we had designed an A/B experiment, how do we measure differences in terms of the measures collected from telemetry?

It seems tempting just to compare the quality measures for both groups, say the average rate of corrections to meeting minutes. However, since observations are noisy it is worth analyzing experimental results with the proper statistics.

For example, we might see that 15% of 2,158 users in the control group with an old summary model correct the meeting minutes but only 10 percent of the twenty users in the treatment group do—but given the small sample size, we should have little confidence in the 33 percent improvement, which may just be due to random noise.

In a nutshell, the more samples we have in each group and the larger the difference between the group averages, the easier it is to be confident that an observed difference is not due to random noise. With large enough groups, even small differences can be detected reliably and confidently.

In practice, the *t-test* is a common statistical test to check whether two samples are likely from the same distribution, that is, whether we observe an actual difference between the treatment and control group rather than just random noise. The *t-test* technically has a number of assumptions about the data, but is robust if the samples are large, which they are almost always when performing A/B testing.

Be careful: *t-test* can be only applied if we have two concurrent groups.

Software Engineering for Artificial Intelligence: Testing in Production

Testing and Experimenting in Production: A/B Experiments - Minimizing Exposure

Just some few additional notes. Typically, we want to assign as few users to the treatment group as necessary, in case the experiment fails and we expose those users to lower qualities of service, or even risks.

At the same time, we need a certain number of participants to gain confidence in the results, and the smaller the effect size we try to observe, the more participants we need.

In theory, **power statistics** provide some means to compute needed sizes. In practice, most people running A/B tests will develop some intuition, usually use a somewhat smallish percent of their user base (say 0.5 to 10 percent) for the treatment group, and run experiments for a while until they see statistically significant results.

It is fairly straightforward to set up a custom A/B experiment from scratch by introducing a few feature flags, a hard-coded user-variant mapping, adjusting telemetry, and maybe adding a t-test. However, it is often a good idea to invest in some infrastructure to make experimentation easy and flexible to encourage data scientists and software engineers to experiment frequently and make evidence-based decisions and improvements based on experiment results. This is described as **continuous experimentation**.

Many companies offer ready-to-use infrastructure to make experimentation easy. At their core, many of these services manage feature flags and provide a dashboard to show experimental results. They typically integrate functionality to automatically schedule experiments and dynamically adjust experiment parameters. Examples are [launchdarkly](#), [split.io](#), and [Flagsmith](#).

Software Engineering for Artificial Intelligence: Testing in Production

Testing and Experimenting in Production: Canary Releases



Coal miners face many constant dangers: cave-ins, explosions, fires, and dangerous gases like carbon monoxide. The gas is odorless, colorless, and tends to replace oxygen molecules in the bloodstream, which keeps actual oxygen molecules from reaching organs and tissues.

Canaries, it turns out, are *much more sensitive* to carbon monoxide and other poisonous gases than humans. Around 1911, miners started carrying canaries into the mines with them, and they quickly became a metaphor for warning signs – when the canary keels over, it's time to evacuate the mine before you become the next victim.

"*The canary in the coal mine*" is an idiom that means something or someone that warns of danger.

Software Engineering for Artificial Intelligence: Testing in Production

Testing and Experimenting in Production: Canary Releases

Canary releases use the same infrastructure as A/B experiments but focus on **deployment**. Where A/B experiments analyze an experimental change to see whether it improves the product, canary releases are usually designed as a safety net for releasing intended changes.

Canary release. A software deployment strategy where a new version of a system (e.g., an AI model or service) is rolled out *gradually* to a small subset of users before a full release. This allows practitioners to *monitor performance and detect issues* in a controlled environment, minimizing risk.

The idea is to treat every release like an *experiment* that is initially only rolled out to a small number of users (treatment group), to limit exposure in case the release performs poorly. If the new release performs worse than the previous release according to telemetry (with all the statistical tests of A/B experiments), it is rolled back so that all users return to the control group and receive the previous release. If the release performs similarly or better, it is further deployed to more users. Typically, a canary release starts with internal users, beta users, and a very few normal users (under 1 percent), and is then increasingly deployed to 5 percent and then 20 percent of all users before it is released to all.

Technically, canary releases and A/B experiments usually share the same infrastructure.

Software Engineering for Artificial Intelligence: Testing in Production

Testing and Experimenting in Production: Other forms of testing in production

A/B experiments and canary releases are the most common forms of experimentation in production, but they are far from the only ones.

Shadow releases run two variants in parallel, but show only the predictions of the previous system or model to the user.

This way, users are *not yet affected* by the change, but we can observe whether the new variant is stable. When testing models with shadow execution, we can observe how often the new model agrees with the old model. If we can gain some ground truth labels for the production data, we can even compare the accuracy of the two variants.

Shadow releases are traditionally used to test the stability, latency, and resource consumption of a system in production settings.

Blue/green deployment is a name for experiments where both the old and the new variant of a system or model are deployed and a load-balancer or feature flag immediately switches all traffic to the new version.

In contrast to incremental deployments in canary releases, the new variant is released to everybody at the same time. Blue/green deployments avoid potential consistency challenges from running different variants in parallel and exposing different users to different variants, but they also expose all users to potential problems.

Software Engineering for Artificial Intelligence: Testing in Production

Testing and Experimenting in Production: Other forms of testing in production

A final, maybe most daring, step of testing in production is *chaos engineering*.

Chaos engineering is the practice of *intentionally introducing failures or unpredictable conditions* into a system to test its *resilience and robustness*.

Chaos engineering is mainly a form of testing robustness, popularized by Netflix for testing their infrastructure built on Amazon's cloud infrastructure.

To see how the whole system reacts to specific outages, Netflix's automated tool deliberately takes down servers and observes through telemetry whether overall system stability or user experience was affected—for example, it may measure how many users' Netflix stream was interrupted after deliberately taking down a server.

While it initially may seem crazy to deliberately inject faults in a production system, this strategy instills a culture for developer teams to anticipate faults and write more robust software (e.g., error recovery, failover mechanisms) and it allows robustness testing of distributed systems at a scale and realism that is difficult to mirror in a test environment.

The idea can naturally be extended to AI components, especially when testing infrastructure robustness for questions such as “*would system quality suffer if this model would be 10 percent less accurate or have 15 percent more latency?*”.

Chaos engineering may, however, affect service-level agreement; so, it needs to be designed responsibly.

Software Engineering for Artificial Intelligence: Testing in Production

Testing and Experimenting in Production: Experimenting Responsibly

Experimenting in production can be extremely powerful once an active user base is available, but it can also expose users to risks from releasing poor-quality systems or models. Experiments can fail, releases can break functionality, and injected small faults of a chaos experiment can take the entire service down.

A key principle is always to minimize the *blast radius*, i.e., to plan experiments such that the consequences of bad outcomes are limited. Usually, it is prudent to test changes offline before exposing them to users and even then to restrict the size of the treatment group to the needed minimum. When it comes to chaos experiments, designers usually think carefully about how to isolate effects to few users or subsystems.

For the same reason, it is important to be able to *quickly change deployments*: for example, abort a release or redirect all traffic of the treatment group back to the original variant. This relies on a well-designed and well-tested infrastructure for operations, based on DevOps principles and tools, such as automation, rigorous versioning, containerization, and load balancers.

In this context, *automation* becomes essential to act quickly on insights. Automation can also make experimentation more efficient, as it can terminate experiments once confident results are collected and then immediately start the next experiment.

Whenever experimenting in production, we should always be aware that we are experimenting on users, real humans, with real consequences. Even with minimizing the blast radius and quick rollbacks, experiments can cause real harm to users.

Software Engineering for Artificial Intelligence: Testing in Production

Before leaving... What Should You Remember?

Software Engineering for Artificial Intelligence: Testing in Production

Testing and Experimenting in Production

With model validation and testing, we still refer to **offline** validation and verification—that is, the evaluation of models using held-out datasets and input testing strategies *prior* to deployment.

This process allows us to assess correctness in a *controlled/artificial setting*. However, such testing is inherently limited: it assumes that future inputs will resemble those seen during offline settings. To address this gap, we now move toward **testing in production**, which aims to *monitor and assess model behavior under real-world conditions*, where unexpected inputs, data drift, or unseen failure modes may emerge.

Testing in production. The process of evaluating a model's behavior after deployment, using real-world inputs and conditions. It complements offline validation by capturing issues like data drift, unexpected inputs, or performance degradation that only emerge in live environments.

The overarching idea is to “try” ideas directly in production multiple times a day to see what works, receiving real, representative data from the real environment.

Testing in production can avoid the common pitfalls of offline model evaluations, such as test data not being representative of production data. However, testing in production also raises new challenges, since **quality problems may be directly exposed to users**, potentially causing real harm.

Software Engineering for Artificial Intelligence: Testing in Production

Testing and Experimenting in Production: Canary Releases

Canary releases use the same infrastructure as A/B experiments but focus on **deployment**. Where A/B experiments analyze an experimental change to see whether it improves the product, canary releases are usually designed as a safety net for releasing intended changes.

Canary release. A software deployment strategy where a new version of a system (e.g., an AI model or service) is rolled out *gradually* to a small subset of users before a full release. This allows practitioners to *monitor performance and detect issues* in a controlled environment, minimizing risk.

The idea is to treat every release like an *experiment* that is initially only rolled out to a small number of users (treatment group), to limit exposure in case the release performs poorly. If the new release performs worse than the previous release according to telemetry (with all the statistical tests of A/B experiments), it is rolled back so that all users return to the control group and receive the previous release. If the release performs similarly or better, it is further deployed to more users. Typically, a canary release starts with internal users, beta users, and a very few normal users (under 1 percent), and is then increasingly deployed to 5 percent and then 20 percent of all users before it is released to all.

Technically, canary releases and A/B experiments usually share the same infrastructure.

Software Engineering for Artificial Intelligence: Testing in Production

Testing and Experimenting in Production: Other forms of testing in production

A final, maybe most daring, step of testing in production is *chaos engineering*.

Chaos engineering is the practice of *intentionally introducing failures or unpredictable conditions* into a system to test its *resilience and robustness*.

Chaos engineering is mainly a form of testing robustness, popularized by Netflix for testing their infrastructure built on Amazon's cloud infrastructure.

To see how the whole system reacts to specific outages, Netflix's automated tool deliberately takes down servers and observes through telemetry whether overall system stability or user experience was affected—for example, it may measure how many users' Netflix stream was interrupted after deliberately taking down a server.

While it initially may seem crazy to deliberately inject faults in a production system, this strategy instills a culture for developer teams to anticipate faults and write more robust software (e.g., error recovery, failover mechanisms) and it allows robustness testing of distributed systems at a scale and realism that is difficult to mirror in a test environment.

The idea can naturally be extended to AI components, especially when testing infrastructure robustness for questions such as “*would system quality suffer if this model would be 10 percent less accurate or have 15 percent more latency?*”.

Chaos engineering may, however, affect service-level agreement; so, it needs to be designed responsibly.

Software Engineering for Artificial Intelligence: Testing in Production

Testing and Experimenting in Production: A/B Experiments

This is another example of the *Flywheel effect*: The more users, the easier it is to experiment in production, and more experiments help improve the product, which attracts more users.

Controlled experiments in production are called **A/B experiments**.

A/B Experiment (or A/B Testing). A method used to compare two versions of a system, called *Version A* (the control) and *Version B* (the variation), by exposing different user groups to each version and measuring which performs better based on predefined metrics (e.g., accuracy, user engagement, error rate).

In other terms, we simply assign some percentage of the users to a **treatment** group and serve them a different variant of the product, typically differing only in a small change, e.g., trying a different prompt for generating meeting minutes or presenting the meeting minutes with a different layout in a automated minute transcription system.

For example, not all Whatsapp users received a feature allowing them to chat with Meta AI (Llama 3.2): this might be an example of an A/B experiment meant to analyze the user engagement with the feature before distributing it on a large scale.

To implement A/B experiments, we need three ingredients: (1) two or more alternative implementation variants to be tested, (2) a mechanism to map users to those variants, and (3) a mechanism to collect telemetry separately for each variant.

Software Engineering for Artificial Intelligence: Testing in Production

Testing and Experimenting in Production: Other forms of testing in production

A/B experiments and canary releases are the most common forms of experimentation in production, but they are far from the only ones.

Shadow releases run two variants in parallel, but show only the predictions of the previous system or model to the user.

This way, users are *not yet affected* by the change, but we can observe whether the new variant is stable. When testing models with shadow execution, we can observe how often the new model agrees with the old model. If we can gain some ground truth labels for the production data, we can even compare the accuracy of the two variants.

Shadow releases are traditionally used to test the stability, latency, and resource consumption of a system in production settings.

Blue/green deployment is a name for experiments where both the old and the new variant of a system or model are deployed and a load-balancer or feature flag immediately switches all traffic to the new version.

In contrast to incremental deployments in canary releases, the new variant is released to everybody at the same time. Blue/green deployments avoid potential consistency challenges from running different variants in parallel and exposing different users to different variants, but they also expose all users to potential problems.

Software Engineering for Artificial Intelligence: Testing in Production

Testing and Experimenting in Production: Experimenting Responsibly

Experimenting in production can be extremely powerful once an active user base is available, but it can also expose users to risks from releasing poor-quality systems or models. Experiments can fail, releases can break functionality, and injected small faults of a chaos experiment can take the entire service down.

A key principle is always to minimize the *blast radius*, i.e., to plan experiments such that the consequences of bad outcomes are limited. Usually, it is prudent to test changes offline before exposing them to users and even then to restrict the size of the treatment group to the needed minimum. When it comes to chaos experiments, designers usually think carefully about how to isolate effects to few users or subsystems.

For the same reason, it is important to be able to *quickly change deployments*: for example, abort a release or redirect all traffic of the treatment group back to the original variant. This relies on a well-designed and well-tested infrastructure for operations, based on DevOps principles and tools, such as automation, rigorous versioning, containerization, and load balancers.

In this context, *automation* becomes essential to act quickly on insights. Automation can also make experimentation more efficient, as it can terminate experiments once confident results are collected and then immediately start the next experiment.

Whenever experimenting in production, we should always be aware that we are experimenting on users, real humans, with real consequences. Even with minimizing the blast radius and quick rollbacks, experiments can cause real harm to users.



Software Engineering for Artificial Intelligence

Testing and Experimenting in Production

